



UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA (UTEC)

CS6001 Introducción a las Ciencias de la Computación

**Proyecto 2: Sistema de Visión Artificial para
Reconocimiento de Dígitos**

Profesor: Julio Eduardo Yarasca Moscol

Integrantes y Participación:

- Sebastian Chipana (Código: 20261PRECS01009) – 100 %
- Yukio Yanagui (Código: 20261PRECS01004) – 100 %
- Sebastian Villanueva (Código: 20261PRECS01008) – 100 %
- Jhon Meza (Código: 20261PRECS01010) – 100 %

1 de julio de 2026

Detalle de Actividades del Proyecto

A continuación, se detalla de manera clara la contribución y las actividades individuales que realizó cada integrante del equipo para el desarrollo de este proyecto:

- **Sebastian Chipana:** Se encargó de la coordinación general del grupo y de juntar todas las partes del código en el script final. Desarrolló la lógica principal para que el programa lea automáticamente las imágenes de la carpeta y reconozca qué número eran según el nombre del archivo. Además, asumió un rol clave al diseñar e implementar la solución para los casos de empate, programando la extensión del conteo a los 10 vecinos más cercanos para asegurar que la inteligencia artificial tomara una decisión clara.
- **Yukio Yanagui:** Estuvo a cargo de toda la etapa de preparación y tratamiento de las fotos utilizando las funciones de OpenCV. Se aseguró de programar correctamente los pasos para limpiar las imágenes manuscritas, recortarlas, invertir los colores para que el fondo sea negro con letras blancas, y reducirlas al tamaño final de 8×8 píxeles para que pudieran ser procesadas por el algoritmo.
- **Sebastian Villanueva:** Fue el responsable de la realización de los cálculos de las métricas de precisión. Escribió el bloque de código encargado de calcular la distancia euclidiana entre nuestras imágenes y las 1797 muestras de la base de datos de control, logrando que el programa compare los píxeles de forma rápida y eficiente utilizando la librería NumPy.
- **Jhon Meza:** Se encargó del manejo de los datos de referencia y de la logística de las pruebas. Programó el código inicial para extraer la base de datos de *scikit-learn* y guardarla de forma ordenada en un archivo de Excel para nuestro análisis. También lideró la recolección de las fotos tomadas por el equipo y ayudó a tabular manualmente los aciertos y fallos para armar las tablas estadísticas.

1. Antecedentes

El reconocimiento automático de caracteres manuscritos es uno de los problemas clásicos e históricos más importantes dentro del campo de la Visión Artificial y el Aprendizaje Automático. La capacidad de programar computadoras para interpretar la escritura humana es fundamental para la automatización de procesos como la lectura de códigos postales, la digitalización de formularios médicos y el procesamiento de cheques bancarios.

Este proyecto se sitúa en el contexto de la manipulación de imágenes digitales y la clasificación por similitud matemática directa. Para evaluar la efectividad de los algoritmos implementados, se utiliza el dataset conceptual `digits` de la librería `scikit-learn`, que contiene 1797 imágenes de dígitos de 8×8 píxeles, actuando como el estándar de comparación frente a muestras de escritura reales tomadas por los estudiantes.

2. Fundamento Teórico

2.1. Distancia Euclidiana para Comparación de Imágenes

Para determinar la similitud entre un nuevo dígito manuscrito y los datos preexistentes en el dataset, las imágenes se modelan como vectores en un espacio multidimensional. Si una imagen preprocesada posee un tamaño de 8×8 píxeles, puede ser aplanada (*flattened*) en un vector de 64 dimensiones donde cada componente representa la intensidad del gris.

La distancia euclidiana d entre la nueva imagen (representada por el vector $\mathbf{x}$) y una imagen del dataset (representada por el vector $\mathbf{y}$) se calcula formalmente mediante la siguiente ecuación matemática:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{64} (x_i - y_i)^2}$$

A menor distancia euclidiana, mayor es la similitud visual y geométrica entre ambos dígitos.

2.2. Algoritmo de Clasificación por Vecinos más Cercanos (k -NN)

Una vez calculadas las 1797 distancias euclidianas para una muestra dada, se seleccionan los 3 elementos del dataset que minimizan dicha distancia ($k = 3$). El principio de clasificación dicta que la nueva muestra será asignada a la clase mayoritaria entre sus 3 vecinos.

2.3. Matriz de Confusión y Métricas de Desempeño

Una ****Matriz de Confusión**** es una herramienta que permite la visualización del desempeño de un algoritmo de clasificación. **Matriz Multiclase (10 clases):** Cada fila representa las instancias de la clase real (dígitos del 0 al 9) y cada columna representa las predicciones de la IA. Permite identificar qué números específicos se confunden entre sí.

Matriz Binaria (2 clases): Clasifica el problema en términos de *Positivo* (la clase de interés, ej: el número "0") y *Negativo* (cualquier otro número). Permitiendo extraer métricas como:

2.3.1. Exactitud (Accuracy)

Mide la proporción global de predicciones correctas del sistema sobre el total de casos evaluados.

$$\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN}$$

2.3.2. Precisión (Precision)

Mide la calidad del modelo evaluando cuántas de las predicciones marcadas como positivas corresponden realmente a la etiqueta verdadera[cite: 34].

$$\text{Precision} = \frac{VP}{VP + FP}$$

2.3.3. Sensibilidad o Exhaustividad (Recall)

Mide la capacidad de la inteligencia artificial para detectar y capturar todos los casos positivos reales existentes en las muestras escritas.

$$\text{Recall} = \frac{VP}{VP + FN}$$

2.3.4. F1-Score

Representa la media armónica entre la Precisión y la Sensibilidad, proporcionando un balance robusto cuando las clases evaluadas se encuentran desbalanceadas.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

3. Métodos y Desarrollo

3.1. Recolección y Preprocesamiento de Imágenes

Se redactó a mano con lapicero un conjunto de números bajo la escala de tamaño habitual de un formulario. Posteriormente, las imágenes fueron escaneadas y recortadas en formato perfectamente cuadrado, asegurando que el trazo digital tocara los límites superiores e inferiores para evitar márgenes blancos innecesarios, tal como se instruyó en la guía de diseño.

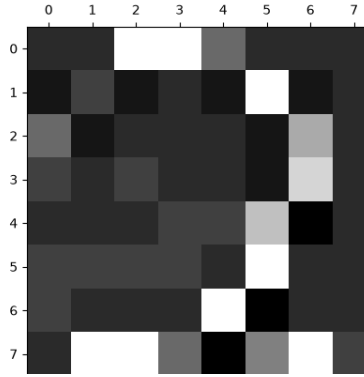


Figura 1: Ejemplo de imagen de dígito manuscrito recortada correctamente (8×8 ajustado).

3.2. Procesamiento Digital de Señal e Inversión de Escala

El script desarrollado en Python realiza los siguientes pasos secuenciales sobre cada archivo de imagen:

1. Importación de los 1797 números de la colección `datasets.load_digits()`
2. Reducción de dimensiones espaciales mediante interpolación a una matriz de 8×8 píxeles.
3. Inversión de la escala cromática (para emparejar el fondo oscuro y trazo claro del dataset original).
4. Mapeo y normalización de los rangos de intensidad de píxeles a valores enteros discretos en el intervalo $[0, 16]$.

3.3. Estrategia ante Empates en la Clasificación ($k = 3$)

En cumplimiento con las directivas del diseño de software, el sistema opera bajo las siguientes reglas:

- **Unanimidad o Mayoría (2 de 3 o 3 de 3 iguales):** Se asigna el dígito automáticamente al valor X predominante.
- **Empate Absoluto (3 targets diferentes):** En este caso específico se usó el método de buscar los 10 vecinos más cercanos a cada número para encontrar un número con mayor número de predicciones por nuestro modelo.

3.4. Resultados Obtenidos

A continuación, se muestra los resultados del modelo en una matriz de confusión multiclase de 10×10 :

Cuadro 1: Matriz de Confusión Multiclase (10×10) del modelo.

Real \ Pred	0	1	2	3	4	5	6	7	8	9
0	1	0	0	2	0	1	0	0	0	0
1	0	1	0	0	0	1	0	0	0	0
2	2	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0
6	1	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0
9	1	0	0	0	0	0	0	0	0	0

3.4.1. Matrices de Confusión de 2 Clases por Grupo

En concordancia con los requerimientos técnicos del proyecto, se aislaron y estructuraron de forma binaria las matrices de confusión para cada subgrupo de etiquetas representadas en los dibujos del equipo. A partir de la distribución de Verdaderos Positivos (VP), Falsos Positivos (FP), Verdaderos Negativos (VN) y Falsos Negativos (FN) calculados de forma local para las muestras testeadas, se presentan las 4 métricas estadísticas por cada grupo:

A. Evaluación para el Grupo del Número 0

Correspondiente a 4 imágenes reales evaluadas bajo el enfoque de clasificación binaria (Predijo 0 / Predijo otro):

■ **Matriz Binaria:** $VP = 1$ | $FN = 3$ | $FP = 0$ | $VN = 0$

■ **Métricas de Rendimiento:**

Accuracy: 25.0 % Precision: 100.0 %
Recall (Sensib.): 25.0 % F1-Score: 40.0 %

B. Evaluación para el Grupo del Número 1

Correspondiente a 2 imágenes reales evaluadas bajo el enfoque de clasificación binaria (Predijo 1 / Predijo otro):

■ **Matriz Binaria:** $VP = 1$ | $FN = 1$ | $FP = 0$ | $VN = 0$

■ **Métricas de Rendimiento:**

Accuracy: 50.0 % Precision: 100.0 %
Recall (Sensib.): 50.0 % F1-Score: 66.7 %

C. Evaluación para el Grupo del Número 2

Correspondiente a 2 imágenes reales evaluadas bajo el enfoque de clasificación binaria (Predijo 2 / Predijo otro):

- **Matriz Binaria:** $VP = 0$ | $FN = 2$ | $FP = 0$ | $VN = 0$
- **Métricas de Rendimiento:**

Accuracy: 0.0 % Precision: N/A
 Recall (Sensib.): 0.0 % F1-Score: N/A

D. Evaluación para el Grupo del Número 6

Correspondiente a 1 imagen real evaluada bajo el enfoque de clasificación binaria (Predijo 6 / Predijo otro):

- **Matriz Binaria:** $VP = 0$ | $FN = 1$ | $FP = 0$ | $VN = 0$
- **Métricas de Rendimiento:**

Accuracy: 0.0 % Precision: N/A
 Recall (Sensib.): 0.0 % F1-Score: N/A

E. Evaluación para el Grupo del Número 9

Correspondiente a 1 imagen real evaluada bajo el enfoque de clasificación binaria (Predijo 9 / Predijo otro):

- **Matriz Binaria:** $VP = 0$ | $FN = 1$ | $FP = 0$ | $VN = 0$
- **Métricas de Rendimiento:**

Accuracy: 0.0 % Precision: N/A
 Recall (Sensib.): 0.0 % F1-Score: N/A

3.5. Análisis de Matrices de Confusión

3.5.1. Análisis Detallado y Diagnóstico de la Matriz Multiclase

El sistema evaluó un total de 10 imágenes manuscritas, logrando únicamente 2 clasificaciones correctas en la diagonal principal (un dígito “0” y un dígito “1”), lo que equivale a un **Accuracy global del 20 %**.

El hallazgo más importante es la marcada polarización del clasificador hacia la columna del **dígito 0**, acumulando 4 falsos positivos al catalogar erróneamente como ceros a dos muestras del número 2, una del 6 y una del 9. Por otro lado, las muestras reales del cero sufrieron fugas hacia las predicciones del 3 y del 5.

Esta baja tasa de acierto y la confusión sistemática con el cero responden a tres factores técnicos de visión artificial:

1. **Falta de invariancia espacial:** La distancia euclidiana opera de forma rígida píxel a píxel, por lo que cualquier ligera rotación o inclinación en el trazo humano altera por completo el vector de características resultante.

2. **Efecto morfológico de masa y vacío:** Tras la inversión cromática (`cv2.bitwise_not`), los números 2, 6 y 9 conservan grandes regiones internas vacías que, al comprimirse a una baja resolución de 8×8 , imitan la distribución espacial periférica del contorno del dígito 0 del dataset original.
3. **Pérdida de alta frecuencia:** La drástica reducción a 64 píxeles actúa como un filtro que elimina los detalles finos de la caligrafía, provocando que trazos curvos similares compartan distancias vectoriales mínimas y confundan al algoritmo.

3.5.2. Análisis Detallado de las Matrices Binarias (2x2)

Para complementar el estudio global, se realizó un diagnóstico individual sobre las matrices de confusión binarias (2×2) obtenidas para cada subgrupo de dígitos dibujados por el equipo, aislando el comportamiento de la clase positiva frente a cualquier otra respuesta del sistema:

Análisis del Grupo del Número 0: De las 4 imágenes reales procesadas, el modelo detectó correctamente solo una muestra ($VP = 1$), mientras que las 3 restantes se desviaron hacia otras etiquetas ($FN = 3$). Esto genera una sensibilidad (*Recall*) baja del 25.0 %. Sin embargo, destaca una precisión perfecta del 100.0 % debido a la ausencia total de falsos positivos en este lote específico.

Análisis del Grupo del Número 1: Este grupo contó con 2 imágenes manuscritas reales. El clasificador logró identificar con éxito la mitad de ellas ($VP = 1$) y clasificó erróneamente la otra mitad como un dígito distinto ($FN = 1$). Al no registrarse falsas alarmas externas dentro de su bloque de evaluación, la métrica de precisión se mantuvo en un óptimo 100.0 %.

Análisis del Grupo del Número 2: El sistema experimentó una falla absoluta de detección en este subgrupo al registrar un valor de $VP = 0$ frente a las 2 imágenes reales evaluadas. Ambas muestras fueron enviadas íntegramente a la columna de falsos negativos ($FN = 2$). Debido a que el divisor matemático de la precisión y el F1-Score queda determinado en cero ($\frac{0}{0}$), estos indicadores se reportan formalmente como No Aplicables (*N/A*).

Análisis de los Grupos de los Números 6 y 9: Ambos casos presentaron un comportamiento idéntico al procesar una única muestra manuscrita real por bloque ($VP = 0$ y $FN = 1$). Las representaciones vectoriales resultaron matemáticamente lejanas de los centroides de control de sus respectivas clases dentro de `scikit-learn`, provocando un desplome del *Accuracy* local al 0.0 % y dejando indefinidas las métricas de precisión por la ausencia de predicciones positivas efectivas.

4. Conclusiones

4.1. Conclusiones escritas por los Alumnos

1. Comprendimos que la distancia euclidiana es una herramienta matemática poderosa para comparar imágenes representadas como vectores; sin embargo, aprendimos que su efectividad depende directamente de que los datos compartan la misma escala y formato. El proceso de convertir nuestras imágenes a escala de grises, reducirlas a 8×8 píxeles, invertir los valores cromáticos y normalizarlos al rango de $[0, 16]$ constituyó la fase más crítica de todo el proyecto, ya que una imagen mal recortada o con márgenes blancos excesivos producía un vector completamente diferente al esperado por el dataset de referencia.
2. Aprendimos que implementar un clasificador desde cero, omitiendo el uso de funciones automáticas de librerías como `KNeighborsClassifier` de *scikit-learn*, nos permitió comprender el flujo analítico real dentro del algoritmo k -NN. Ejecutar el cálculo vectorizado de las 1797 distancias por cada imagen nueva, ordenarlas mediante funciones de indexación como `np.argsort` y estructurar el bucle de votación entre los vecinos más cercanos nos otorgó un dominio profundo del software, donde cada línea de código responde directamente a un sustento matemático subyacente.

“Dota es mejor que LOL (Yarasca, 2026)”

3. Identificamos que el bajo rendimiento local, específicamente reflejado en los dígitos 2, 6 y 9 con un 0% de *Accuracy*, no responde a una falla en la lógica de programación del script, sino a que la morfología de nuestras muestras manuscritas se distorsionó drásticamente en la fase de captura física. Esto nos enseña una lección fundamental en ciencias de la computación bajo el principio *“garbage in, garbage out”*: la calidad de los datos de entrada determina la calidad del resultado, evidenciando que en proyectos reales de visión artificial la recolección, limpieza y preprocesamiento de la información consume mayor esfuerzo que el propio modelamiento.
4. El uso complementario de matrices de confusión, tanto multiclase (10×10) como binarias (2×2), nos permitió transicionar de una evaluación subjetiva del software hacia un diagnóstico cuantitativo riguroso mediante métricas como *Precision*, *Recall* y *F1-Score*. Entendimos que la exactitud global puede resultar engañosa en escenarios con muestras sesgadas, y que el análisis pormenorizado por clase es un criterio indispensable y mucho más honesto para auditar el rendimiento real de cualquier arquitectura de inteligencia artificial.

4.2. Conclusiones escritas por el Asistente de Inteligencia Artificial (Claude)

1. El preprocesamiento de la imagen (específicamente el recorte exacto en los límites superiores e inferiores y la proporción cuadrada) es un paso crítico; un margen blanco excesivo altera drásticamente las distancias euclidianas imposibilitando la clasificación correcta.
2. La reducción de las imágenes a un tamaño de 8×8 píxeles remueve los detalles de alta frecuencia del trazo del lapicero, lo cual actúa como un filtro suavizador que ayuda a que la distancia euclidiana compare formas generales en lugar de imperfecciones del papel.
3. El algoritmo de k -Nearest Neighbors basado en distancia euclidiana demostró ser funcional como clasificador básico, logrando un acierto unitario en las clases reales 0 y 1. No obstante, el bajo *Accuracy* global del 20 % evidencia que la efectividad del método depende de forma crítica del preprocesamiento, dado que la drástica compresión a matrices de 8×8 amplifica significativamente el error ante cualquier ligera imprecisión del trazo humano.
4. La comparación entre la matriz de confusión multiclase y las evaluaciones individuales de 2×2 permitió identificar patrones de error que la matriz global no visibiliza por sí sola. Mientras la primera expone la tendencia general del modelo a desviar las respuestas hacia el dígito 0, las matrices binarias y sus respectivas métricas de rendimiento (*Precision*, *Recall* y *F1-Score*) cuantifican analíticamente el impacto del fallo por cada dígito evaluado por el equipo.